

FreeFalcon 6 — Linux Port: Status Report

Port development log (Claude-assisted)
June 6, 2026

1. Overview

This document describes the state of the native Linux port of **FreeFalcon 6**, an open-source descendant of the *Falcon 4.0* F-16 combat flight simulator (~3,500 source files, ~800k lines of C/C++, originally Windows-only, DirectX 7 era).

Porting strategy. Rather than rewriting subsystems, the port supplies a Windows compatibility layer and keeps the original game code largely intact:

Windows dependency	Linux replacement
Win32 windowing / message loop	SDL2 window + custom message queue (main_linux.cpp)
DirectDraw 7 / Direct3D 7	OpenGL fixed-function backend behind real COM-style DD7/D3D7 vtables (src/compat/d3d_gl.cpp)
DirectSound	OpenAL implementation of the DirectSound interfaces (src/compat/openal_dsound.cpp)
DirectInput	SDL2 keyboard/joystick mapped onto the sim's IO structure and DIK scancodes
Win32 API (files, threads, sync, registry-ish .ini)	src/compat/ headers: pthreads-backed CRITICAL_SECTION/events/threads, case- insensitive file resolution (fopen_nocase), real GetPrivateProfileInt/String
MSVC build	CMake + Ninja, GCC, 64-bit

Repository: /home/g/ff, branch develop, pushed to sim-museum/freefalcon-central. The window title bar shows the git hash the binary was built from, e.g. Free Falcon 6 Linux Port [00725ef1].

2. Current state — what works

All items below are screenshot- or log-verified on the Korea theater data set.

2.1 Core game loop

- Main menu UI at 60 FPS, all screens (Setup, Logbook, Comms, Tactical Engagement

shell, Dogfight, Campaign) load and respond to mouse, including right-click popup menus and double-clicks.

- Campaign/scenario loading (LZSS-compressed .cam files), flight deaggregation, full mission launch pipeline *Menu* → *3D flight* → *exit* → *Menu*, repeatable without restart.
- Clean process exit from the Exit button (exit code 0; previously a 3-stage SIGABRT).

2.2 Flight simulation

- 3D world rendering ~60 FPS: terrain with trilinear mipmapped textures and haze fog, time-of-day lighting, sky, clouds.
- The complete F-16 cockpit: HUD, both MFDs (FCR/HSD pages), DED, RWR, steam gauges, 2-D cockpit and **3-D virtual cockpit** at the correct pilot-eye perspective.
- Radar: APG-68 model scans, paints and tracks targets (RWS verified; beam sweep $\pm 60^\circ$, multi-bar).
- Weapons: gun and missile employment (AIM-9 verified end-to-end; AIM-120 honors its maddog/boresight rules), missile fly-out, impact, and kill removal.
- Particle effects: explosions (fireball/flash/smoke column/debris) and smoke render as proper soft billboards.
- Audio: engine, weapons and UI sounds through OpenAL (verified by recording the PipeWire sink and computing RMS).
- Joystick: axes (remappable via `FF_THROTTLE_AXIS` / `FF_YAW_AXIS`), buttons, POV hat, hot-plug.

2.3 Dogfight mode (multiplayer-style instant arena)

- Full flow: Dogfight screen → saved setup → COMMIT → lobby roster (add AI aircraft per team via right-click popup; entries render with team icon, callsign, type) → TAKEOFF → combat → guns/missile kills.

2.4 Development & test infrastructure

- Reproducible automation: scripted UI clicks (`FF_UI_CLICK`), sim keyboard injection (`FF_SIM_KEY`), scripted view changes + screenshots (`FF_VIEW_SCRIPT`), periodic UI screenshots, auto-launch Instant Action (`-test-ia`).
- ~20 env-gated diagnostic traces (radar, pickle chain, campaign bubble, dogfight slots, texture uploads, GL pixel attribution with backtraces, etc.) — see `CLAUDE.md` for the full table.
- ASAN build variant in `build-asan/`.

3. Notable root causes fixed (the “bug classes”)

These recurring classes account for most port defects; new symptoms should be checked against them first. Each was fixed in multiple places and is documented at the fix sites.

1. **32-bit long in binary file formats.** Windows-written game data uses 4-byte fields; on 64-bit Linux `long/unsigned long` are 8 bytes. Fixed with `int32_t/uint32_t` in every

- loader (resources, campaign saves, WAV, terrain offsets, VU types).
2. **64-bit pointer truncation.** `(int)ptr`, `(DWORD)ptr`, `(GLint)ptr` casts destroyed the upper pointer bits (graphics allocator, texture handles, campaign-object attachment). Fixed with `intptr_t/uintptr_t`.
 3. **DDSURFACEDESC2 read from disk.** The on-disk DDS header is the 124-byte 32-bit layout; the 64-bit struct is 136 bytes — pixel format misread *and* image data offset wrong. Fixed with a dedicated `DDS_FILE_HEADER` in three loaders (texture bank, far-terrain, generic `ReadDDS`).
 4. **MSVC `RAND_MAX` (32767) assumptions.** `rand()`/32767-style scaling produced thresholds thousands of times too large under glibc (radar detection, campaign spotting coin-flips).
 5. **CRLF after `fgets`.** Windows text-mode reads strip `\r`; Linux keeps it, corrupting the last token of every line (particle effect names, `.ini` values).
 6. **Silently-defaulting compat stubs.** `GetPrivateProfileInt/String` stubs zeroed every tuning value in the game (campaign aggregation hysteresis, bubble sizes, AI tuning).
 7. **Signal-less infinite waits / lock-order inversions.** Windows' implicit serialization (`SendMessage`, `_FORCE_MAIN_THREAD`) hid several deadlocks: shutdown AB-BA, campaign thread parked forever in UI mode, and the mission-launch `Camp` ↔ `Vu` critical-section inversion (fixed by enforcing `Camp` → `Vu` ordering).
 8. **OpenGL state-at-call-time vs D3D state-at-draw-time.** `glClear` honors write masks and scissor (D3D's `Clear` does not — the famous “black terrain” bug, and again for stencil); `glLightfv(GL_POSITION)` bakes the current modelview (D3D transforms by `VIEW` at draw); per-vertex D3D emissive (`D3DMCS_COLOR2`) has no fixed-function GL equivalent (emulated with `glMaterial` inside `glBegin/End`); DXT1 must be the RGBA variant to keep D3D's 1-bit punch-through alpha.

4. What remains to be done

4.1 Known open defects

- **#10 — Terrain visible through 3-D-pit MFD screens and lower panel gaps.**
Diagnosed to the frame's *last* polylist flush painting far-terrain tiles into the pit's chroma-keyed screen holes (the instrument background quad draws earlier with depth writes off). The full per-pixel attribution toolkit is in place; the remaining question is which mechanism protects those pixels on Windows (instrument depth writes, flush ordering, or stencil over the chroma holes). Tracked in `CLAUDE.md` §Open Issues with reproduction commands.
- **Stale window-title hash** — the embedded git hash only refreshes when CMake reruns, so the title can lag the actual binary. Cosmetic but has repeatedly confused testing.
- **Intermittent campaign-thread SIGSEGV** in `AS_DataClass::ASSearch` (ground-unit `A*` pathfinding) seen rarely during Instant Action. Not yet reproduced under a debugger.

4.2 Untested / unexercised areas

- **Campaign mode proper** (the strategic war): loads and launches, but long-running war simulation, time compression UI, mission planning UI, and debrief flow have had no systematic testing.
- **Tactical Engagement** (scripted missions) and the TE mission editor.
- **Multiplayer** (networked dogfight/campaign): the VU networking layer compiles and the local loopback path works; real two-machine play is untested.
- **ACMI recording/playback, night ops** (NVG path exists but unverified), **weather states** beyond fair, helicopters, the full ground war (artillery, SAM engagement logic).
- **Other theaters** (Israel, EuroWar data trees exist; EuroWar uses a newer save version).

4.3 Choice points (decisions that shape future work)

1. **Renderer evolution: stay fixed-function vs. shader path.** The compat layer emulates D3D7 through OpenGL 1.x-style immediate mode. It works, but per-vertex emissive emulation, COMBINE-env juggling and CPU-side vertex submission cost performance and complexity. *Options:* (a) keep fixed-function (lowest risk, current course); (b) move the compat layer to VBO + small shader set (eliminates several gotcha classes, large but mechanical); (c) adopt a wrapper like dxvk-native style translation (heavyweight; D3D7 is poorly served by existing wrappers).
2. **Exit-time strategy.** The port currently calls `_exit(0)` after orderly cleanup to skip the legacy static-destructor minefield. *Alternative:* fix each destructor — more “correct,” low practical value. Decide whether `_exit` is acceptable permanently.
3. **Data compatibility layer.** All loaders now parse 32-bit Windows binary formats in-place. *Alternative:* a one-time data converter to native-endian/64-bit-clean formats (faster loads, simpler code, but forks the data and breaks drop-in use of existing installs).
4. **Threading model.** Several Windows-era races were fixed point-wise (lock ordering, explicit signals). A deeper refactor (single render thread + message passing) would remove the class entirely; the current model is now stable but fragile to new code.
5. **Packaging & distribution.** Game *data* is not redistributable with the engine. Decide on: bare instructions (current), an installer script that ingests a user-supplied FreeFalcon/Falcon install, or Flatpak/AppImage with a data-import step.
6. **Upstreaming.** The port lives on develop of a fork. Decide whether to propose merging to the parent FreeFalcon project (requires keeping the Windows build green — the `#ifdef FF_LINUX` discipline so far was designed for this) or to maintain a Linux fork.
7. **Performance budget.** Currently ~60 FPS at 1024×768 on the dev machine. If higher resolutions/refresh rates are a goal, profiling (likely texture upload paths and immediate-mode submission) becomes worthwhile, and interacts with choice (1).

5. Building and running

5.1 Prerequisites

- Linux x86-64 (developed on Ubuntu-family, GCC ≥ 13, CMake ≥ 3.22, Ninja).

- SDL2, GLEW, OpenAL development headers. On the dev machine these are **not** system-installed; they are extracted into extern/usr (gitignored). To recreate:

```
cd /home/g/ff/extern
apt-get download libsdl2-dev libglew-dev libglew2.2 libopenal-dev
for d in *.deb; do dpkg -x "$d" . ; done
```

- Game data: a FreeFalcon 6 installation (e.g., from an old Wine prefix). Dev machine path: /home/g/sgl/SAT/freeFalcon/WP/drive_c/FreeFalcon6. The data dir needs a sim -> Zips/sim symlink (zip contents pre-extracted; the resource-manager zip path is bypassed on Linux).

5.2 Build

```
cd /home/g/ff/build          # CMake already configured with -G Ninja (Release)
ninja                      # incremental build; full build ~570 targets
# ASAN variant:
cd /home/g/ff/build-asan && ninja
```

First-time configure, if ever needed:

```
cmake -S /home/g/ff -B /home/g/ff/build -G Ninja -DCMAKE_BUILD_TYPE=Release
```

After adding source files that use Windows-cased #includes, regenerate the case-alias symlinks: python3 /home/g/ff/fix_include_case.py (run from the repo root; converges in two passes).

5.3 Run

```
# Easiest: the launcher script (also exposed as a desktop entry "FreeFalcon 6
(Linux Port)")
/home/g/ff/run-freefalcon.sh          # normal windowed session
/home/g/ff/run-freefalcon.sh -test-ia # auto-launch Instant Action
(testing)
```

```
# Equivalent manual invocation:
cd /home/g/sgl/SAT/freeFalcon/WP/drive_c/FreeFalcon6
/home/g/ff/build/src/ffviper/FFViper -d "$PWD" -w
```

-d <dir> points at the game data; -w selects windowed mode (1024×768).

In-sim keys (defaults): 1 HUD-only, 2 2-D cockpit, 3 3-D virtual cockpit, Shift-6 padlock; dogfight flow is *Dogfight* → *saved setup* → *COMMIT* → *right-click the roster column to add aircraft* → *TAKEOFF*.

Environment gotcha: if the process dies abruptly (crash/kill — not normal exit), GNOME's mutter-x11-frames can wedge and the next launch hangs on a blank window. Heal with `kill mutter-x11-frames` (it restarts automatically).

5.4 Debugging and test hooks (selection)

Env var	Effect
FF_UI_CLICK="x,y@sec[d r];..."	Scripted UI clicks (1024×768 UI coords; d=double, r=right)

Env var

FF_VIEW_SCRIPT="m@sec;s@sec;..."

FF_SIM_KEY="dik@sec[+holdms];..."

FF_TEST_EXPLOSION=1

FF_DEBUG_RADAR=1

FF_DEBUG_DF=1

FF_DEBUG_BUBBLE / _PICKLE / _MSLEND /
_SFXTEX / _VIEW / _FLUSHES

FF_PROBE_PIXEL="x,y" (+FF_PROBE_BT=1,
FF_DUMP_GLTEX=<id>)

Effect

Scripted view modes + screenshots to
/tmp/ff_view_N.bmp

Inject DIK key events into the sim

Spawn a test explosion ahead of the player
every 2 s

Player FCR scan list, beam state, per-crossing
detection rolls

Dogfight add-aircraft chain end-to-end

Campaign bubble, weapon release, missile end,
SFX atlas, pit camera, polylist flushes

Per-draw pixel attribution with GL state,
backtraces, texture dumps

The authoritative, always-current list lives in `CLAUDE.md` (repo root), which also carries the full session-by-session engineering log and the open-issues tracker.